Fragen: E2E-Testing

1. Mit Blick auf die Test-Pyramide: Warum sollten wir viel weniger E2E-Tests haben im Vergleich zu Unit-Tests?

A:

1. Geschwindigkeit

- **Ein Unit-Test läuft in Millisekunden**
- **Ein E2E-Test kann Sekunden bis Minuten dauern**
- **Bei 1.000 Tests: Unit → ~10 Sek. | E2E → Stunden**

2. Kosten & Infrastruktur

- **E2E braucht Browser, Datenbanken, externe Services, Netzwerk**
- **Unit-Tests laufen isoliert, ohne Setup-Overhead**

3. Fehlerlokalisation

- **Unit-Test schlägt fehl → man weiß *sofort* wo das Problem ist**
- **E2E-Test schlägt fehl → Fehler könnte überall im Stack liegen (UI, API, DB, Netzwerk...)**

4. Flakiness (Instabilität)

- **E2E-Tests scheitern oft aus nicht-funktionalen Gründen: Timing-Probleme, Netzwerklatenz, Browser-Rendering**
- **Unit-Tests sind deterministisch und reproduzierbar**

5. Wartungsaufwand

- **Jede UI-Änderung kann dutzende E2E-Tests brechen**

- **Unit-Tests sind entkoppelt und robuster gegen Refactoring**

  6. Testabdeckung
- **Edge Cases lassen sich mit Unit-Tests leicht abdecken**
- **Jeden Edge Case per E2E zu testen wäre prohibitiv teuer**

2. Welche spezifischen Arten von Bugs kann ein E2E-Test (mit einem echten Browser wie Chromium) abfangen, die isolierte Komponententests mit einem virtuellen DOM übersehen würden?

   A:

   1. Echtes Browser-Rendering & CSS
- **Layout-Bugs: Ein Element ist laut DOM vorhanden, aber durch z-index, overflow: hidden oder opacity: 0 unsichtbar oder nicht klickbar**
- **CSS-Konflikte: Styles aus verschiedenen Quellen überschreiben sich gegenseitig – im virtuellen DOM existiert kein echtes Rendering**
- **Font/Media-Queries: Responsive-Verhalten bei echter Viewport-Größe**

---

2. JavaScript-Engine-Verhalten
- **Browser-spezifische JS-Bugs: Bestimmte APIs verhalten sich in Chromium, Firefox oder Safari unterschiedlich**

- **Timing & Race Conditions: Echte asynchrone Events (Animationen, Transitions) die in jsdom schlicht ignoriert werden**
- **Web APIs: localStorage, IndexedDB, WebSockets, Clipboard API – diese sind in jsdom nur rudimentär simuliert**

---

3. Netzwerk & HTTP

- **Echte API-Calls: Falsche HTTP-Headers, CORS-Fehler, falsche Status-Codes – in Unit-Tests alles gemockt**
- **Cookie/Session-Handling: Werden Cookies korrekt gesetzt, mitgesendet, und nach Logout gelöscht?**
- **Redirect-Flows: Wird nach Login wirklich auf die richtige Seite weitergeleitet?**

---

4. Formulare & Browser-Verhalten

- **Autofill: Browser-Autofill überschreibt Felder und triggert ggf. keine React-Events**
- **Native Validation: required, type="email" – das echte Browser-Popup erscheint nur im echten Browser**
- **File Upload: Echter Datei-Dialog, Drag & Drop**

---

5. Drittanbieter-Integrationen

- **OAuth/SSO-Flows: Der Redirect zu Google/GitHub und zurück kann nur real getestet werden**

- **Payment-Widgets (z.B. Stripe iframe): Externe iframes existieren im virtuellen DOM nicht**
- **Analytics/Tag Manager: Werden Events wirklich gefeuert?**

---

6. Performance-bedingte Bugs

- **Lange Ladezeiten: Ein Spinner verschwindet nie, weil ein API-Call hängt**
- **Memory Leaks: Führen nach mehreren Navigationen zu echten Abstürzen**

3. Was ist „Test Flakiness“ (Tests, die zufällig mal bestehen und mal fehlschlagen), und warum sind E2E-Tests besonders anfällig dafür?
   A:
   Was ist Flakiness?
   Ein flaky Test ist ein Test, der ohne Codeänderung manchmal grün und manchmal rot ist – also nicht-deterministisch.
   Das ist besonders gefährlich, weil das Team lernt, roten Tests nicht mehr zu vertrauen – und echte Bugs dadurch übersehen werden.

Warum sind E2E-Tests besonders anfällig?
1. Timing & Asynchronität
Das häufigste Problem: Der Test klickt auf einen Button, bevor die Seite fertig geladen hat.
javascript

// ❌ Flaky: Element vielleicht noch nicht da
cy.get('#submit').click();

// ✅ Besser: Explizit warten
cy.get('#submit').should('be.visible').click();
```

Netzwerklatenz, CPU-Last und Animationen variieren bei jedem Testlauf.

---

#### 2. Externe Abhängigkeiten
E2E-Tests sprechen mit **echten Systemen**:
- Datenbank antwortet mal langsam, mal schnell
- Externe API ist kurz nicht erreichbar
- Auth-Token läuft genau während des Tests ab

Unit-Tests haben das nicht – alles ist gemockt und deterministisch.

---

#### 3. Geteilter Zustand / Test-Reihenfolge
```

Test A erstellt User → Test B erwartet leere DB → ❌ Schlägt fehl

Wenn Tests nicht sauber isoliert sind, beeinflusst ein Test den nächsten – aber nur manchmal, je nach Ausführungsreihenfolge.

---

4. Browser & Rendering

- **Animationen und CSS-Transitions sind nicht abgeschlossen**
- **Der Browser rendert Elemente zu unterschiedlichen Zeitpunkten**
- **Scroll-Position stimmt nicht, Element ist (noch) außerhalb des Viewports**

---

5. Parallelisierung
In CI/CD laufen Tests oft parallel:

- **Mehrere Tests greifen gleichzeitig auf dieselbe DB zu**
- **Race Conditions zwischen Test-Prozessen**
- **Shared Resources (Ports, Dateien) blockieren sich gegenseitig**

E2E-Tests orchestrieren viele bewegliche Teile gleichzeitig – Browser, Netzwerk, Datenbank, externe APIs. Jede dieser Schichten kann nicht-deterministisch sein. Unit-Tests haben nur eine Schicht: die Logik selbst.

4. Anstatt harcodierte sleep(5000) Pausen zu verwenden, um auf das Laden einer Seite zu warten – wie gehen moderne Frameworks wie Playwrite oder

Cypress mit dem Warten auf Elemente um (Auto-Waiting)?

A:

Das Problem mit sleep()

javascript

// ❌ Hartcodierte Pause – immer falsch

await page.click('#submit');

await sleep(5000); // Zu lang auf schnellen Rechnern, zu kurz unter Last

await page.click('#next');

` ` `

- Zu kurz → Test schlägt fehl obwohl alles okay ist

- Zu lang → Test-Suite dauert unnötig lange

- **Nicht deterministisch – funktioniert auf deinem Rechner, schlägt in CI fehl**

Das Konzept: Auto-Waiting

Moderne Frameworks warten **automatisch darauf, dass ein Element einen bestimmten Zustand erreicht – bevor sie eine Aktion ausführen.**

` ` `

Framework fragt intern (~alle 100ms):

"Ist das Element sichtbar? Ist es klickbar? Ist es stabil?"

→ Nein → warte kurz → frage erneut

→ Ja → führe Aktion aus ✅

→ Timeout überschritten → Test schlägt fehl ❌

Playwright: Auto-Waiting

Playwright wartet bei jeder Aktion automatisch auf mehrere Bedingungen gleichzeitig:

javascript

```
// Playwright wartet automatisch bis der Button:
// - im DOM existiert
// - sichtbar ist
// - nicht disabled ist
// - keine Animation läuft
// - stabil positioniert ist (nicht mehr bewegt)
await page.click('#submit'); // ← kein sleep nötig!
```

Playwright prüft automatisch ("Actionability Checks"):

Check	Bedeutung
visible	Element ist nicht versteckt
stable	Element bewegt sich nicht mehr
enabled	Nicht disabled
editable	Für Eingabefelder
receives events	Kein anderes Element liegt darüber

Explizit warten wenn nötig:

javascript

```
// Auf spezifischen Zustand warten
await
expect(page.locator('#status')).toHaveText('Fertig');
await
expect(page.locator('#spinner')).toBeHidden();

// Auf Netzwerk-Request warten
await page.waitForResponse('/api/users');

// Auf Navigation warten
await page.waitForURL('/dashboard');
```

Cypress: Retry-Mechanismus

Cypress funktioniert etwas anders – es wiederholt Assertions automatisch bis sie erfüllt sind oder ein Timeout erreicht wird:

javascript

```
// Cypress wiederholt diese Assertion bis zu 4
Sekunden lang
cy.get('#submit')        // sucht Element (retry)
  .should('be.visible')   // prüft Sichtbarkeit (retry)
  .click();               // klickt erst wenn beides erfüllt

// Auf Text warten
cy.get('#status').should('contain', 'Fertig');
```

```
// Auf API-Response warten
cy.intercept('GET', '/api/users').as('getUsers');
cy.wait('@getUsers'); // wartet auf echten Netzwerk-Call
```

Vergleich der Ansätze

	Playwright	Cypress
Strategie	Actionability Checks vor jeder Aktion	Retry der gesamten Assertion-Chain
Netzwerk	waitForResponse()	cy.intercept() + cy.wait()
Timeout	Konfigurierbar, default 30s	Konfigurierbar, default 4s
Explizites Warten	Selten nötig	Selten nötig

Wann braucht man doch explizites Warten?

```javascript
// Playwright: Warten auf komplexe Bedingungen
await page.waitForFunction(() => {
  return document.querySelectorAll('.item').length > 10;
});
```

```
// Wenn ein visuelles Element erst nach einer Animation klickbar ist
await expect(locator).toBeVisible();
await expect(locator).not.toBeInViewport(); // erst scrollen
```

Kernaussage

sleep() wartet blind eine feste Zeit. Auto-Waiting wartet intelligent auf den richtigen Zustand. Das macht Tests schneller, stabiler und weniger flaky – weil sie mit der Anwendung synchronisiert sind statt gegen sie zu kämpfen.